

Tide

A partially active market maker with a fee-bounded virtual curve

Team Tide · Version 0.2 · September 26, 2026 · github.com/asengupta07/tide

Abstract. A constant-product pool quotes a stale price until someone trades, and the first trader of each block is usually an arbitrageur who takes the whole pool at that price. Tide exposes only a fraction λ of a maker’s inventory to that trade, quotes the block’s later trades against a curve N times deeper than the exposed slice, and bounds how far that deeper curve can be pushed by a drift threshold δ . We derive the steady-state loss-versus-rebalancing of the design, $1/(2 - \lambda)$ of a plain pool, and confirm it by simulation to three digits. We then show that the deeper curve is a rebate paid by the maker: with no fee it can be drained every block by a round trip that needs no price information, and neither λ , N nor δ bounds the loss. A flat fee f does, provided $(N - 1)\delta \leq 2f$; the contracts enforce this inequality on every parameter change. The same arithmetic library prices two venues, a linch Aqua SwapVM program and a Uniswap v4 hook, and a cross-venue test asserts identical fills. Parameters are ENS text records mirrored on-chain; a manager agent moves them on its own inside guardrails the owner set, and beyond them only after the owner completes a fresh World ID authentication.

1 Introduction

Loss-versus-rebalancing (LVR) is the cost a liquidity provider pays for quoting a price that is stale between blocks [1]. For a constant-product pool it accrues at $\sigma^2/8$ of pool value per unit time; at 60% annualised volatility that is about 4.5% of capital per year before fees. Deployed mitigations either price the arbitrage (dynamic or auctioned fees [2]), remove intra-block ordering (batch clearing) or rebate part of the loss. None of them changes what the arbitrageur can see.

Two recent proposals do. Ko’s partially active market maker [3] exposes only a fraction of the reserves to each block; Kim and Park’s collateralised liquidity scaling [4] quotes uninformed flow against a virtual curve deeper than the reserves that back it. Tide combines both, adds the drift and solvency guards the combination needs, and, in the course of implementing it, finds and closes an economic hole in the virtual curve that neither proposal addresses.

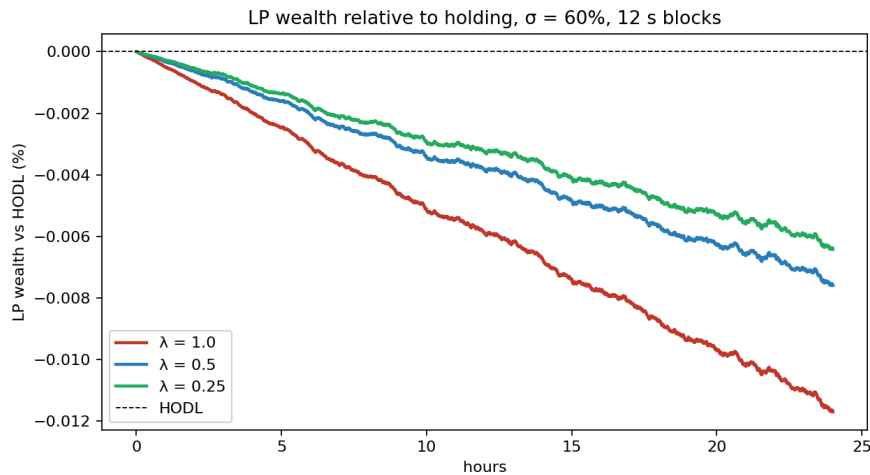


Figure 1: LP wealth relative to holding, one trading day, $\sigma = 60\%$, 12-second blocks, 500 paths, no fees. The fully active pool ($\lambda = 1$) loses LVR every block; exposing half or a quarter of the reserves per block cuts the loss.

2 Model

The market price P_t follows a geometric Brownian motion with volatility σ ; blocks have length Δ . A constant-product pool with reserves (x, y) has marginal price $P = y/x$ and value $E = xP + y$. For a curve of value V the expected per-block loss to an arbitrageur who trades it to the new price is, to first order in Δ ,

$$\text{LVR} = \frac{\sigma^2}{8} \cdot V \cdot \Delta. \quad (1)$$

Tide has three governed parameters and a fee. $\lambda \in (0, 1]$ is the fraction of reserves exposed per block; $N \geq 1$ the virtual depth multiplier; δ the drift bound; f a flat fee on the input token, set by the owner and tied to N and δ by Proposition 3.

3 Mechanism

Per block and per strategy:

1. **Active split.** On the first quote or fill of a block, `active = lambda * total` for each token. The remainder is passive: it stays in the maker's wallet (or the hook's reserves) and is not quoted in this block. Neither venue has a block-boundary callback, so the split is lazy: it happens inside the first quote of the block and is persisted by the first fill.
2. **First fill.** The first fill of a block is priced on the plain constant-product curve over the active reserves ($N = 1$). The arbitrageur sees λ of the inventory.
3. **Later fills.** Every later fill is priced on $(Nx_a)(Ny_a) = k$ over the current active reserves (x_a, y_a) . A trade of size q sees slippage of order $q/(Nx_a)$ instead of q/x_a .
4. **Drift bound.** The reserves right after the first fill are the block's anchor. A later fill that would move the virtual price outside $[P_{\text{anchor}(1-\delta)}, P_{\text{anchor}(1+\delta)}]$ is re-priced on the active curve.
5. **Solvency.** Any fill must be deliverable from active plus passive. A fill that dips into the passive part triggers a re-split from the new totals at the current marginal price, which creates no arbitrage.
6. **Fee.** Every fill pays f on the input token. The curve and the drift guard see the net input; the taker pays gross; the fee rests in the passive part until the next re-split. Both venues read f from the same parameter record as λ , N and δ , so a fill always pays the fee its parameters were checked against.

4 Analysis

Proposition 1 (activeness scales LVR). A strategy of equity E that exposes λE per block loses, in steady state,

$$\mathbb{E}[\text{LVR}_{\text{Tide}}] = \frac{\sigma^2}{8} \cdot E \cdot \Delta \cdot \frac{1}{2 - \lambda}. \quad (2)$$

Proof sketch. Let g_t be the log gap between the market and the pool's marginal price at the top of block t . The first fill closes the gap on the active slice; after the re-split the pool price is the reserve-weighted mix of the active part (at the market) and the passive part (at the old price), so the gap carried forward is $(1 - \lambda)g_t + r_{t+1}$ with $r_{t+1} \sim N(0, \sigma^2 \Delta)$. Thus g is AR(1) with stationary variance $\sigma^2 \Delta / (\lambda(2 - \lambda))$. Closing a gap g on a curve of value λE costs $\lambda E g^2 / 8$ to second order, and taking expectations gives Equation 2. The gap dynamics are those of [3], Section 3. $\square$

The saving is 33% at $\lambda = 0.5$ and 43% at $\lambda = 0.25$; it is capped at 50% as $\lambda \rightarrow 0$, at which point the pool no longer tracks the market. Section 5 confirms the ratio numerically.

Proposition 2 (virtual depth and solvency). Quoting against $(Nx_a)(Ny_a) = k$ reduces the slippage of a trade of size q from about q/x_a to about $q/(Nx_a)$ (Table 2). The virtual curve promises up to Ny_a , which the pool does not hold. With the drift bound measured against the block anchor, the most it can be asked to deliver in one block is

$$y_{\max} = Ny_a(1 - \sqrt{1 - \delta}), \quad (3)$$

so the strategy is solvent whenever $y_a + B \geq y_{\max}$, with B the passive buffer. At $N = 4$, $\delta = 0.2\%$ the bound is 0.4% of y_a , far inside the active slice; the solvency check remains as a hard invariant regardless.

Proposition 3 (the virtual curve is a rebate; the fee bound). A fill on the virtual curve moves the virtual price by some amount and moves the real active reserves by the same tokens, which on the real curve is N times that price move. After a within-band fill the real active price therefore sits up to $(N - 1)\delta$ from the virtual one, and the next block's first fill collects the gap. Every unit of execution improvement the virtual curve grants is a unit of the maker's inventory at the next re-split; inside the band the improvement over the active curve is at most $(N - 1)\delta/2$ per unit traded.

With $f = 0$ this rebate is extractable at will. Let a taker be first in the block. On the active curve she sells x to move the price by $\delta' \geq \delta$; on the virtual curve she buys x back up to the guard. A sale on a shallow curve followed by a purchase on a deep one nets, to first order,

$$(N - 1)N\delta^2/4 \quad (4)$$

of one side of the active reserves per block, with no price gap and no information, and leaves the real reserves $(N - 1)\delta$ mispriced for the following arbitrageur besides. Simulated with the contract arithmetic on a USD 2 million pool at $\lambda = 0.5$, $N = 4$, $\delta = 0.5\%$, the loop takes about USD 28 per block, roughly 830 times the LVR that λ saves in that block at $\sigma = 60\%$. Neither λ , N nor δ bounds the sum over blocks.

A flat fee on every input does. The round trip pays f twice and gains at most the rebate; honest one-directional flow pays f once and the maker keeps it. The rebate cannot be farmed if it never exceeds the fee that pays for it:

$$(N - 1)\delta \leq 2f. \quad (5)$$

The contracts enforce Equation 5 at initialisation and on every change to λ , N , δ or f . Numerically the round trip stays unprofitable up to $4f/(N - 1)$, so the bound keeps a factor of two, and the largest admissible one-directional flow leaves the maker whole after the following arbitrage. A test performs the round trip on the Aqua venue and asserts that the attacker loses and the maker's inventory is worth no less at the old price.

The bound also settles the stale-anchor question. A dust first fill anchors the block at the pool's current price, which arbitrageurs keep within f of the market plus one block of drift, $\sigma\sqrt{\Delta}$ (3.7 bp at $\sigma = 60\%$ and 12 s). A follower on the virtual curve gains at most the gap less f less $\delta/2$, so once δ exceeds two one-block moves the stale anchor is worth nothing. The manager therefore keeps

$$3\sigma\sqrt{\Delta} \leq \delta \leq \frac{2f}{N - 1}, \quad (6)$$

lowering N when the interval is empty. At $\sigma = 60\%$, $f = 30$ bp, $N = 4$ this is $11 \text{ bp} \leq \delta \leq 20 \text{ bp}$; the deployed strategy uses 20 bp. $\square$

Choosing λ . Exposing less of the reserves lowers LVR but lets the pool's token weight w_t drift from its target $\theta = 1/2$; the deviation is AR(1) with decay $(1 - \lambda)$ and shock $r_t/4$. The maker minimises

$$\text{LVR}(\lambda) - \text{Fees}(\lambda) + \kappa \sum_t (w_t - \theta)^2, \quad (7)$$

where the fee term is first order in f and κ prices tracking error. LVR and tracking error scale with σ^2 and fees with σ , so λ^* falls as volatility rises. We solve Equation 7 on a grid at $f = 1$ bp with κ calibrated to $\lambda^* = 0.5$ at $\sigma = 60\%$ (Figure 2); the manager reads λ^* from this frontier and δ from Equation 6.

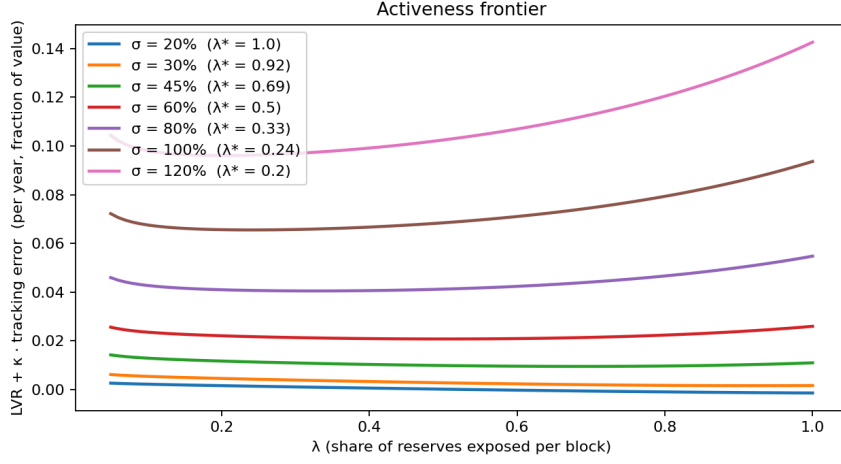


Figure 2: Objective Equation 7 against λ for several volatilities. λ^* is 1.0 at $\sigma = 20\%$, 0.5 at 60%, 0.24 at 100%.

5 Simulation

10,000 GBM paths, $\sigma = 60\%$, 12-second blocks, one day; the arbitrageur trades the active slice to the new price every block.

λ	LVR per day	simulated ratio	$1/(2 - \lambda)$	saved
1.00	0.01233%	1.000	1.000	0%
0.50	0.00822%	0.667	0.667	33.3%
0.25	0.00705%	0.571	0.571	42.9%

Table 1: LVR per day as a fraction of capital. $\lambda = 1$ reproduces $\sigma^2/8/365 = 0.01233\%$.

N	slippage, trade of 1% of active	relative to $N = 1$
1	0.990%	1.0×
2	0.498%	2.0×
4	0.249%	3.97×
8	0.125%	7.9×

Table 2: Slippage of a follow-on trade on the virtual curve.

6 Implementation

One Solidity library, `TideMath`, holds the split, the N -scaled quotes, the drift test, the solvency bound, the fee arithmetic and the parameter box Equation 5. It is checked against an integer Python reference on 24 vectors, and both venues import it.

1inch Aqua. Tide is a SwapVM program of three custom opcodes in unallocated slots, `ACTIVE_SPLIT` (0x92), `VIRTUAL_XYC` (0x52) and `BUFFER_GUARD` (0x22), dispatched by a router that is the swap-vm template with the opcode set extended. Instruction order is security-critical; every Tide opcode scans the program and reverts unless the three appear once each in that order with nothing else but `Salt`, so no control-flow opcode can route around the guard. The maker’s inventory never leaves the wallet: Aqua records balances at `ship` and pulls only at fill time, which is the natural home for a design in which the pool does not hold what the virtual curve promises.

Uniswap v4. The same steps run inside `beforeSwap` of a `BaseCustomCurve` hook that returns a `BeforeSwapDelta` for the whole amount, holds its reserves as ERC-6909 claims, mints pro-rata shares

and refuses liquidity changes in any block that already saw a swap. A cross-venue test runs one trade sequence on both venues and asserts identical amounts.

Parameters. `TideParams` stores λ , N , δ and f per strategy, keyed by the Aqua order hash or the v4 pool id, together with owner-set guardrails for the manager: a λ range, the largest λ move per write, the largest N and a cooldown between writes. Keys are claimed through the venues only (the app checks the caller is the order’s maker; the hook claims its pool id for its owner at initialisation), so nobody can squat a predictable key. The owner’s writes are unbounded; a manager write outside the guardrails reverts; every write is checked against Equation 5.

Path	gas	vs. plain
plain constant-product fill on the same router, swap only	57,402	–
Tide fill, first of block (re-split, fee, state write)	77,234	+19,832
Tide fill, later in block (virtual curve, guard, fee)	76,241	+18,839

Table 3: Gas measured in Foundry, `taker.swap` alone with the input pre-minted. The overhead is one parameter read per opcode, the block state, one balance read and the program scan.

The suite has 53 tests: vector parity, every opcode, two swaps in one block, the lazy re-split, re-pricing of an informed-sized follow-on trade, exact-output beyond inventory, buffer top-up, fee netting, quote-equals-fill in both directions and modes, program-order reverts, governance and guardrails, the round trip of Proposition 3, hook liquidity guards and cross-venue parity.

The reference deployment is on Sepolia (router `0x65a2...a31e`, parameters `0x1685...72aD`, hook `0x1391...AA88`) with a WETH/USDC strategy shipped at $\lambda = 0.5$, $N = 4$, $\delta = 20$ bp, $f = 30$ bp (the manager has since moved λ and δ with volatility), one filled Aqua order and one hook swap in which the quote equalled the fill. A fork script reproduces the flow against the mainnet Aqua registry with real WETH and USDC.

7 Governance

The parameters of a strategy are text records on an ENS name the maker owns (`eth-usdc.tide.eth` for the reference strategy), so anyone can read them and the chain of custody is public. A manager agent with its own name, `manager.tide.eth`, holds a resolver role scoped to exactly the `lambda`, `N` and `delta` records and is the `manager` of the on-chain parameters; it cannot touch the fee, the guardrails, the strategy hash, addresses or the name itself, and the owner revokes it with one call per record.

The agent computes λ^* from the frontier and δ from Equation 6 at the realised volatility. Inside the owner’s guardrails it writes the records and the on-chain parameters on its own: the rule is deterministic and the contract bounds what the key can do, so no human is needed per change. A change outside the guardrails is where authority matters, and there the owner must complete a fresh World ID authentication (`prompt=login`, `max_age=0`); the ID token is validated server-side and its pairwise subject must match the one the owner bound to the wallet, which itself required a signed message from that wallet. The agent then writes the records, and the owner’s wallet applies the change on-chain, since the contract refuses the manager. Denied, expired, replayed and forged responses leave everything unchanged, and a harness asserts each path. Both paths have run on the reference deployment: a change inside the guardrails applied by the agent alone, and one outside them approved with a fresh proof and applied by the owner.

8 Limitations

- The model assumes the block’s first fill is the informed one. A retail order that happens to be first pays the shallow curve. Proposition 3 removes the profit from taking that slot deliberately, not the cost of taking it by accident.

- The frontier’s fee term is first order and solved at 1 bp. At the deployed 30 bp the fee changes when arbitrage happens, since the gap must exceed f first, which the LVR model does not capture; λ^* is the LVR-versus-tracking optimum, not a fee-calibrated one.
- The lower edge of Equation 6 has a proven safe side and a heuristic target; the optimal band for a given mix of flow is open.
- Parameters are mirrored on the trading chain by the agent after approval; a cross-chain read of the record would remove that step.
- Mainnet deployment, audit and multi-pair rebalancing are out of scope for this version.

References

- [1] J. Milionis, C. C. Moallemi, T. Roughgarden, and A. L. Zhang, “Automated Market Making and Loss-Versus-Rebalancing,” *arXiv preprint arXiv:2208.06046*, 2022.
- [2] A. Adams, C. C. Moallemi, S. Reynolds, and D. Robinson, “am-AMM: An Auction-Managed Automated Market Maker,” *arXiv preprint arXiv:2403.03367*, 2024.
- [3] S. Ko, “Partially Active Automated Market Makers,” *arXiv preprint arXiv:2602.09887*, 2026.
- [4] H. Kim and Y.-S. Park, “Bounding LVR in AMMs via Secant-Tangent Divergence and Collateralized Liquidity Scaling,” *arXiv preprint arXiv:2605.19267*, 2026.